extremely flexible and potentially complex flows. Typical CI jobs include running tests, packaging multiple modules, and copying artifacts.

Common CI systems include Jenkins (jenkins-ci.org/), Hudson (hudson-ci.org/), Bamboo (atlassian.com/software/bamboo), TeamCity (www.jetbrains.com/teamcity/), and Travis CI (travis-ci.org/). Although each has slightly different features, they all do the same thing, which is take a change set from your SCM system, run any build scripts you have in your codebase, and output the result somewhere.

A bit about Jenkins

Jenkins, formerly called Hudson, is an open source CI system that is very popular, has a plethora of plugins for additional functionality, is extremely easy to set up and administrate, and has a very active community. When Hudson became commercial software, the open source version continued to evolve as Jenkins. Because Jenkins is by far the most popular build system, we'll be using it for our examples in this book. If you're not familiar with it and want more information, check out the Jenkins home page: jenkins-ci.org/. Note that everything we do with Jenkins in this book could easily be applied to other CI systems.

Build scripts are the brain of your CI process. A build script is code that tells your CI system how to compile your software, put it together, and package it. Your build script is essentially the instructions that your CI system should follow when it integrates code. Common build frameworks are Ant, Nant, Ivy, Maven, and Gradle. Build scripts are powerful tools that manage dependencies, determine when to run tests and what tests to run, and check for certain conditions in a build to determine whether it should continue or stop and return an error. Figure 5.4 shows graphically what a typical build script will control.

Figure 5.4. A build script running on a CI server controlling a multistep build process